

Dataset Pruning: 영향으로 학습 데이터를 줄이는 법

Dataset Pruning: Reducing Training Data by Examining Generalization Influence

Shuo Yang, Zeke Xie, Hanyu Peng, Min Xu, Mingming Sun, Ping Li (2023)

개별 점수 대신 제거 집합의 공동 영향을 제한하는 데이터 선택 방법

학습 데이터의 중복이 반복 계산 비용을 키운다

큰 데이터셋의 모든 예제가 같은 정도로 일반화에 기여하지는 않는다

- 데이터가 커지면 학습·튜닝·구조 탐색을 반복하는 비용도 함께 커진다
- 예제를 하나씩 빼고 재학습하면 영향은 알 수 있지만, 실제 규모에서는 불가능하다
- 개별 점수만 정렬하면 여러 예제가 함께 만드는 상쇄와 증폭을 놓친다
- 목표는 성능 손실을 제한하면서 남길 학습 집합을 가능한 작게 만드는 것이다

Dataset pruning은 데이터 개수를 먼저 정하지 않고 허용 가능한 변화에서 출발한다

데이터를 줄이는 세 가지 질문

선택 예산, 합성 데이터, 허용 변화는 서로 다른 문제를 만든다

접근

점수 기반 선택

dataset condensation

dataset pruning

고정 제거량 variant

먼저 고정하는 것

남길 데이터 수

합성 예제 수

허용 parameter 변화

제거할 예제 수 m

찾는 것

점수가 높은 예제

새로 만든 proxy data

제거할 최대 중복 집합

영향이 가장 작은 집합

논문의 새 설정은 cardinality 제약을 generalization 관련 제약으로 바꾼다

개별 영향이 커도 함께 제거하면 상쇄될 수 있다

영향은 scalar 순위가 아니라 방향을 가진 vector로 더해야 한다

강의용 vector	개별 norm	함께 더한 결과
$g_1 = (3, 0)$	3	첫 vector
$g_2 = (-3, 0)$	3	둘째 vector
$g_1 + g_2$	개별 값은 큼	$(0, 0)$, norm 0

gradient norm이나 influence norm만 정렬하면 이 joint effect를 볼 수 없다

문제에 등장하는 집합과 모델

제거 집합과 남은 집합을 먼저 분리하면 최적화 식을 읽기 쉽다

기호	뜻	역할
D, n	원본 dataset과 예제 수	pruning 출발점
$\hat{D}, m$	제거할 subset과 크기	최대화 대상
$\tilde{D} = D - \hat{D}$	남은 training set	최종 재학습 data
$\hat{\theta}$	D 에서 얻은 ERM 해	parameter 기준점
$\hat{\theta}_{-\hat{D}}$	$\hat{D}$ 제거 후 ERM 해	변화량 비교 대상

논문은 남길 집합이 아니라 제거할 중복 집합 $\hat{D}$ 를 직접 선택한다

ε -redundant subset의 정의

제거 전후 parameter 거리가 ε 이하이면 그 subset을 중복으로 본다

조건

$$||\hat{\theta} - \hat{\theta} - \hat{\theta}||_2 \leq \varepsilon$$

읽는 법

model parameter 변화 제한

주의

test accuracy 식이 아님

$\hat{D}_\varepsilon$

조건을 만족한 제거 subset

여러 개가 가능

$\hat{D}_\varepsilon \max$

가장 큰 ε -redundant subset

남은 $\tilde{D}$ 는 가장 작아짐

parameter 근접성이 generalization 근접성을 대신하는 핵심 proxy다

정확한 subset 검사는 2^n 번 재학습을 요구한다

가능한 제거 집합마다 모델을 다시 학습하는 방식은 확장되지 않는다

01 n 개 예제 각 예제를 제거하거나 유지



02 2^n 개 subset 가능한 조합을 모두 생성



03 매번 재학습 $\hat{\theta}$ - $\hat{D}$ 를 새로 계산



04 영향 근사 한 기준 model에서 변화 추정

Influence Function은 subset마다 재학습하는 비용을 한 번의 local 근사로 바꾼다

한 예제의 제거 영향을 재학습 없이 근사한다

sample gradient를 Hessian inverse로 보정해 parameter 이동 방향을 얻는다



Eq. (2)는 trained solution 주변의 1차 local approximation이다

Influence Function이 계산하는 것과 계산하지 않는 것

parameter 이동의 1차 추정치를 실제 재학습 결과와 같은 값으로 보면 안 된다

구분	논문의 값	경계
입력	$\hat{\theta}$ 에서의 gradient와 Hessian	한 local 지점
출력	parameter change vector	test metric 직접 예측 아님
subset	개별 influence의 합	선형 근사
실험 구현	마지막 linear layer만 계산	전체 network와 다름

계산이 빨라지는 대신 local linearization과 layer 근사를 받아들인다

Subset 영향은 선택한 vector의 합이다

S는 각 예제 제거의 parameter 변화 근사를 모아 둔 행렬이다

기호	정의	해석
s_i	$-l_{\text{param}}(z_i) / n$	예제 i 제거 vector
S	모든 s_i 를 모은 matrix	candidate influence 집합
$W_i = 1$	예제 i를 $\hat{D}$ 에 선택	유지가 아니라 제거
$W^T S$	선택 vector의 합	joint parameter 변화

최적화 변수 W 는 예제별 순위를 만들지 않고 조합을 직접 고른다

Eq. (3): 허용 영향 안에서 제거 수를 최대화한다

목적 함수와 constraint를 분리해 읽으면 선택 방향이 선명해진다

구성	표현	뜻
변수	$W \in \{0,1\}^n$	각 예제 제거 여부
목적	maximize $\sum W_i$	제거 example 최대화
constraint	$ W^T S _2 \leq \epsilon$	joint influence 제한
결과	$\hat{\epsilon}_{\max} = \{z_i : W_i=1\}$	가장 큰 feasible 제거 집합

ϵ 가 작을수록 parameter 변화는 더 제한되지만 제거 가능한 수는 줄 수 있다

Eq. (4): 제거 수를 고정하면 영향 norm을 최소화한다

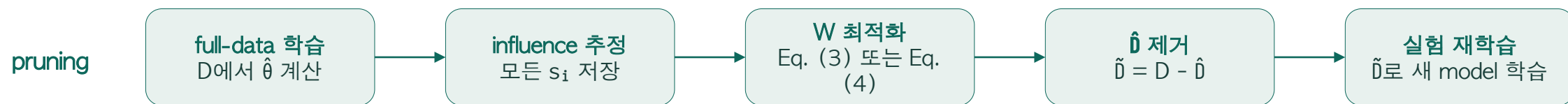
실험 비교에서는 baseline과 같은 pruning ratio를 맞추기 위해 이 variant를 쓴다

질문	Eq. (3)	Eq. (4)
먼저 정하는 값	영향 tolerance ϵ	제거 수 m
최적화 목표	$\sum W_i$ 최대화	$ W^T S _2$ 최소화
공통 변수	binary selector W	binary selector W
논문 실험	theory 검증에 연결	baseline pruning 비교

같은 pruning ratio의 성능 비교가 Eq. (3)의 tolerance-driven 문제와 같지는 않다

Algorithm 1과 이후 평가의 흐름

전체 model을 한 번 학습한 뒤 influence와 discrete solver로 제거 집합을 만든다



pruning 전 full-data 학습과 influence 계산이 필요하므로 선택 단계도 공짜가 아니다

강의용 최적화 예: 큰 두 vector가 함께 사라진다

$\varepsilon = 0.2$ 에서 세 removal vector 중 feasible한 최대 집합을 찾는다

선택	합 vector	norm	판정
{1}	(0.6, 0)	0.6	불가
{2}	(-0.6, 0)	0.6	불가
{1,2}	(0, 0)	0	가능
s_3 포함 조합	(0,0.7) 또는 $(\pm 0.6, 0.7)$	0.7 이상	불가

이 toy에서 최대 feasible 제거 집합은 {1,2}이며 cardinality는 2다

Theorem 1 : expected loss gap의 상계

논문은 influence constraint를 loss gap과 연결하는 asymptotic bound를 제시한다

항목	표기·관계	의미
gap	$ L(\theta_{\text{after}}) - L(\theta_{\text{before}}) $	제거 후·전 expected test loss 차이
premise	$ \sum l_{\text{param}}(z_i) _2 \leq \varepsilon$	unscaled influence 합 제한
bound	$O(\varepsilon / n + m/n^2)$	1차 항과 2차 remainder
derivation	Taylor + Cauchy-Schwarz	local approximation 사용

상계는 observed accuracy가 ε 만큼만 감소한다는 직접 보장이 아니다

Guarantee를 읽을 때 남겨야 할 조건

논문의 이론, 근사, 구현은 같은 수준의 주장이 아니다

- Hessian은 positive definite라고 가정한다
- Influence Function과 subset 합은 $\hat{\theta}$ 주변의 1차 근사다
- Eq. (3)의 S 는 이미 $1/n$ 을 포함하지만 theorem premise는 unscaled 합을 쓴다
- 실험은 full network가 아니라 마지막 linear layer의 influence를 계산한다
- simulated annealing은 discrete problem의 global optimum을 보장하지 않는다

일반화 bound는 이 조건을 포함한 분석 결과이며 모든 pruning 실행의 절대 보증이 아니다

Figure 2: influence와 관측 loss gap

논문은 pruning size에 따라 ϵ/n 과 실제 test loss 차이가 함께 변하는지 확인한다

비교 항목	Figure 2의 관찰	해석 범위
theory proxy	ϵ/n curve	m/n^2 항은 생략
observed gap	$L(\theta_{\text{after}}) - L(\theta_{\text{before}})$	제거 후·전 empirical test loss 차이
optimization-based	두 curve가 비슷한 scale	exact equality는 아님
random pruning	더 큰 ϵ 와 gap	이 실험 조건의 비교

Figure 2는 theorem을 대신하는 증명이 아니라 approximation의 empirical sanity check다

실험 조건

결과를 읽기 전에 dataset, network와 training recipe를 고정한다

항목	조건	범위
dataset	CIFAR-10/100, TinyImageNet	50k, 50k, 100k images
network	1.25M, ResNet18, ResNet50	11.69M, 25.56M
training	200 epochs, batch 128	SGD, cosine decay
optimizer detail	lr 0.01, momentum 0.9	weight decay 5×10^{-4}
pruning solver	last-layer influence	simulated annealing

동일 hyperparameter로 retraining하지만 influence와 solver는 approximation이다

Figure 1의 비교는 같은 pruning ratio에서 수행한다

Optimization-based 방법은 high pruning ratio에서 baseline과 더 벌어진다

비교군	선택 기준	joint effect
Random-Herding	무작위·class center	직접 고려 안 함
Forgetting·GraNd·EL2N	학습 이력·loss 관련 score	개별 score
Influence-score	개별 influence norm	방향 상호작용 누락
Optimization-based	subset influence norm	vector 합으로 고려

Figure 1은 Eq. (4)로 제거량을 맞춘 empirical comparison이다

Pruned dataset은 더 큰 network에서도 평가된다

작은 network로 CIFAR-10을 prune하고 ResNet18-50을 새로 학습한다

단계	조건	해석
pruning model	1.25M small network	source label 불일치
target model	ResNet18 11.69M	unseen larger architecture
target model	ResNet50 25.56M	unseen larger architecture
Figure 3	CIFAR-10 accuracy curves	transfer 가능성 관찰

§6.1은 SqueezeNet, Figure 3과 §6.4는 SENet이라 적어 재현 전 확인이 필요하다

Figure 4: pruning ratio와 convergence의 trade-off

CIFAR-10 곡선의 표시점은 시간 감소와 accuracy 감소를 함께 보여 준다

조건	convergence time	test accuracy	full 대비
whole dataset	약 3200 s	95.1%	기준
40% pruning	약 1800 s	93.8%	0.56×, -1.3 pp
60% pruning	약 1250 s	92.3%	0.39×, -2.8 pp

40% pruning의 절반 수준 시간과 1.3 pp 하락은 이 Figure 4 조건의 관찰값이다

NAS에서 작은 proxy dataset을 반복 사용한다

Pruning 비용을 한 번 지불하고 많은 architecture training을 줄이는 사용 사례다

01	CIFAR-10 proxy	class당 100장, 총 1000장
----	----------------	----------------------



02	720 ConvNets	각 proxy에서 100 epochs
----	--------------	----------------------



03	ranking 비교	validation Spearman correlation
----	------------	---------------------------------



04	선택 architecture	full-data 성능으로 평가
----	-----------------	-------------------

반복 trial이 많을수록 proxy data의 준비 비용을 나눠 부담할 수 있다

Table 1 : CIFAR-10 NAS proxy 결과

같은 1000-image proxy끼리 비교하면 proposed subset의 ranking이 가장 가깝다

Dataset	성능	Spearman ρ	시간	images
Random	79.4%	0.21	113 min	10^3
Herding	80.1%	0.23	113 min	10^3
Forgetting	82.5%	0.79	113 min	10^3
Ours	85.7%	0.94	113 min	10^3
Whole	85.9%	1.00	3029 min	5×10^4

Ours는 113 min과 1000 images에서 85.7%· $\rho = 0.94$ 를 보고한다

Dataset pruning의 비용과 적용 경계

작은 dataset의 이득을 얻기 전에 full model과 influence 계산이 필요하다

항목	필요한 비용	해석 제한
기준 model	full D에서 먼저 학습	one-shot 절감 아님
influence	모든 예제 vector 저장	last-layer approximation
Hessian	inverse estimate 계산	positive-definite 가정
subset solver	simulated annealing	global optimum 미보장
외부 적용	새 task에서 재검증	image/NAS 결과에 한정

많은 model을 같은 data에서 반복 학습할 때 비용 상쇄 가능성이 가장 크다

정리: 제거 집합의 공동 영향으로 data를 줄인다

Dataset pruning은 tolerance-driven subset selection을 influence optimization으로 본다

- 한 sample의 제거 효과를 Hessian-adjusted influence vector로 근사한다
- 여러 sample의 vector 합을 제한해 group effect를 선택 과정에 포함한다
- CIFAR와 NAS 실험은 계산 절감과 accuracy trade-off를 조건부로 보여 준다
- Theorem의 scale, Taylor assumption, last-layer 구현과 heuristic solver를 함께 읽어야 한다

핵심 기여는 좋은 예제 순위가 아니라 함께 제거해도 되는 집합을 찾는 문제 설정이다